01

BASES
DE DONNÉES
MODERNES

Apprendre
Comprendre
Construire
demain

CHAPITRE 2

Partie 4
Modélisation des données en MongoDB

Concevoir des documents et des collections adaptés aux besoins de l’application.

Introduction à la modélisation documentaire

Penser autrement que le modèle relationnel

SQL

Réflexe relationnel

Entités → Tables → Relations → Normalisation

DB

Réflexe MongoDB

Besoins de l'application → Documents → Collections → Relations

- **Question centrale**
  Quelles données doivent être stockées ensemble ?
- **Critère principal**
  La manière dont l'application consulte et modifie les données
- **Objectif**
  Réduire les requêtes inutiles tout en gardant un modèle cohérent

Deux stratégies principales

Embedding ou References

{} Embedding — imbrication

Les **données** liées sont **stockées dans le même document.**

→ References — références

Les **données** sont **stockées dans des documents séparés.**

Embedding

```
{ nom: "Sara",
  adresse: { ville: "Casablanca" } }
```

References

```
// étudiant
{ nom: "Sara", formationId: 10 }
// formation
{ _id: 10, nom: "Informatique" }
```

i | Le choix ne dépend pas seulement de la cardinalité : il dépend surtout des usages de l'application.

Quand utiliser l'Embedding ?

Stocker ensemble les données fortement liées

Produit + caractéristiques

```
{
  nom: "PC Portable",
  caracteristiques: {
    processeur: "Intel Core i7",
    ram: "16 Go",
    stockage: "512 Go SSD"
  }
}
```

- **Les données appartiennent au même objet**
  ex. un produit et ses caractéristiques
- **Elles sont souvent consultées ensemble**
  une seule lecture récupère le document complet
- **Elles sont modifiées ensemble**
  cohérence plus simple au niveau du document
- **Elles ne grandissent pas sans limite**
  éviter les tableaux non bornés

¡ | **Question à se poser :** cette donnée appartient-elle seulement à ce document ? Si oui, on l'imbrique.

Quand utiliser les References ?

Séparer les données qui ont leur propre existence

Étudiant → formation

```
// étudiant
{ _id: 1, nom: "Sara",
formationId: 10 }

// formation
{ _id: 10, nom: "Génie
informatique" }
```

- **Données partagées**
  plusieurs étudiants ont la même formation
- **Données consultées seules**
  la formation existe indépendamment
- **Données volumineuses**
  éviter de grossir le document principal
- **Relation complexe ou N–N**
  références souvent plus adaptées
- **Duplication difficile à maintenir**
  ne pas recopier partout la même donnée

i | **Règle simple :** utiliser les References lorsque les données ont leur propre cycle de vie.

Modéliser les relations

1–1, 1–N et N–N dans un modèle documentaire

1 Relation 1–1

Produit → caractéristiques
Imbrication adaptée

N Relation 1–N

Commande → produits
Tableau de sous-documents si liste limitée

N Relation N–N

Étudiants ↔ cours
Références adaptées, sauf quand l'autre côté n'est qu'une simple valeur qui ne change pas (ex: compétences)

Exemple 1–N par imbrication

```
{ numero: "CMD01",
  produits: [
    { nom: "Clavier", qte: 1 },
    { nom: "Souris", qte: 2 }
  ]}
```

¡ La cardinalité donne une indication, mais le choix final dépend des requêtes, de la taille et de l'évolution des données.

Normalisation et dénormalisation

Comprendre la différence avec le modèle relationnel

SQL

Relationnel

On cherche souvent à éviter les répétitions en séparant les données dans plusieurs tables.

DB

MongoDB

On peut parfois regrouper ou dupliquer certaines données pour faciliter la lecture.

Exemple dénormalisé

```
{ numeroCommande: 1001,
  client: { nom: "Sara", ville:
"Casablanca" } }
```

- **Équilibre à trouver**
  performance, cohérence, taille, duplication
- **À ne pas confondre**
  dénormaliser ≠ tout mettre dans un seul document

TP — Modéliser un campus simplifié

Appliquer les critères d'imbrication et de référence

? **Mme Bennani :** *« Avant de tout informatiser, proposez un modèle MongoDB pour le campus. »*

Le besoin métier

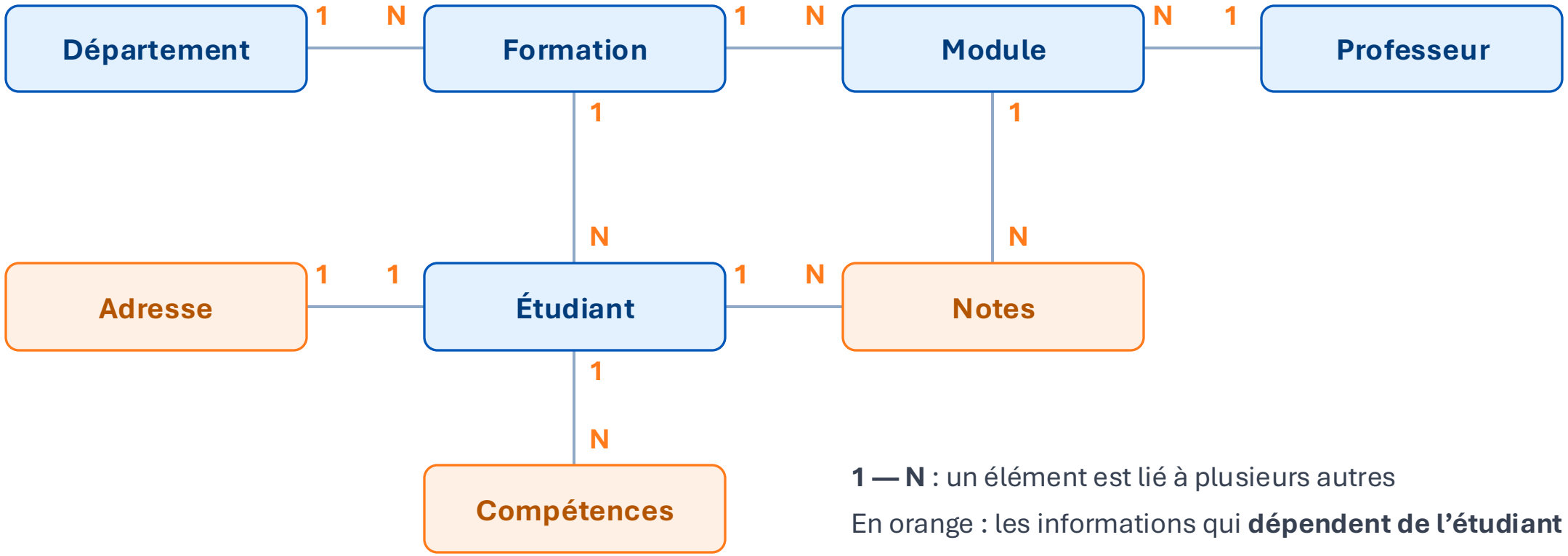
- Un **département** gère plusieurs **formations**.
- Une formation accueille de nombreux **étudiants** ; la liste grandit chaque année.
- Chaque étudiant a **une adresse** et une courte liste de **compétences**.
- Chaque étudiant a ses **notes** : une par module suivi.
- Une formation propose plusieurs **modules**.
- Un **professeur** enseigne plusieurs modules.

À rendre

1. Les **entités** et leurs **relations**, avec les cardinalités (1–1, 1–N, N–N).
2. Pour chaque relation : **imbrication** ou **référence** ? Justifiez avec les critères vus.
3. Un **exemple de document** pour la collection `etudiants`.

Correction — question 1 : les relations

Les entités du campus et leurs cardinalités



1 — N : un élément est lié à plusieurs autres

En orange : les informations qui **dépendent de l'étudiant**

Correction — imbriquer ou référencer ?

Le choix pour chaque relation et sa justification

Relation	Choix	Critère
Étudiant → adresse	**Imbriquer**	appartient à un seul étudiant
Étudiant → compétences	**Imbriquer**	liste courte, propre à l'étudiant
Étudiant → notes	**Imbriquer**	nombre limité, lues avec l'étudiant
Formation → étudiants	**Référencer**	la liste grandit chaque année
Département → formations	**Référencer**	partagé par plusieurs formations
Note → module	**Référencer**	module partagé par tous les étudiants
Module → professeur	**Référencer**	professeur partagé par plusieurs modules

Question 3 : un étudiant

```
{ _id: 1001,
  nom: "Sara Alaoui",
  adresse: { ville:
"Rabat" },
  competences: ["Java",

"MongoDB"],
  notes: [
    { moduleId: 200,
      note: 15 } ],
  formationId: 10 }
```

Appartient et reste court → imbriquer. **Partagé**, autonome ou qui grandit → référencer.

Département : imbriquer ou référencer ?

Pourquoi le département devient une référence dans les formations

Imbriqué dans chaque formation

```
{ nom: "Génie logiciel",
  departement: { nom: "Informatique",
                 chef: "M. Berrada" } }
{ nom: "Data Science",
  departement: { nom: "Informatique",
                 chef: "M. Berrada" } }
{ nom: "Réseaux",
  departement: { nom: "Informatique",
                 chef: "M. Berrada" } }
```

Le même département est **recopié** 3 fois.

Nouveau chef : modifier **toutes** les formations.

Un département **sans formation** est impossible.

Référencé dans les formations

```
// departements
{ _id: 1, nom: "Informatique",
  chef: "M. Berrada" }

// formations
{ nom: "Génie logiciel", departementId: 1 }
{ nom: "Data Science", departementId: 1 }
{ nom: "Réseaux", departementId: 1 }
```

Le département est stocké **une seule fois**.

Nouveau chef : **une seule** modification.

Le département existe **indépendamment**.

i | Critère : le département est **partagé** par plusieurs formations → référence.

CHAPITRE 2

Partie 5
Optimiser, analyser et sécuriser

Index, explain(), pipeline d’agrégation et validation JSON Schema.

Les index : trouver vite

Éviter de parcourir toute la collection à chaque requête

Commandes de base

```
// Créer un index sur le statut
db.orders.createIndex({ status: 1 })

// Lister les index de la collection
db.orders.getIndexes()

// Supprimer un index par son nom
db.orders.dropIndex("status_1")
```

- **Sans index**
  MongoDB lit **tous** les documents : **`COLLSCAN`**.
- **Avec index**
  une liste **triée** des valeurs mène directement aux bons documents : **`IXSCAN`**.
- **1 ou -1**
  ordre croissant ou décroissant de l'index.
- **Nom de l'index**
  champ_ordre, par exemple **`status_1`**.
- **_id**
  toujours indexé automatiquement.

i Un index **accélère les lectures** mais **ralentit les écritures** et occupe de l'espace : on indexe les champs souvent **filtrés** ou **triés**.

Index composé et index unique

Adapter l'index aux requêtes fréquentes

Index composé : historique d'un client

```
// 1. L'index
db.orders.createIndex(
  { customerId: 1, createdAt: -1 })

// 2. exemple de requête qu'il accélère
db.orders.find({ customerId: 2 })
         .sort({ createdAt: -1 })
```

Index unique : un e-mail par client

```
db.customers.createIndex(
  { email: 1 }, { unique: true })
```

La règle du préfixe

Requête	Index ?
`{ customerId: 2 }`	oui
`{ customerId: 2 }` + tri sur `createdAt`	oui
`{ createdAt: {...} }` seul	non

- **Pourquoi ?**
  l'index est trié d'abord par **`customerId`**, puis par **`createdAt`** : il ne sert que si la requête utilise le **premier champ**.
- **Index unique**
  un doublon est refusé avec l'erreur **`E11000 duplicate key`**.

Analyser une requête avec explain()

Voir comment MongoDB a cherché, sans changer le résultat

Requête analysée

```
db.orders.find({ customerId: 2 })
         .explain("executionStats")
```

Sans index

```
stage: COLLSCAN
totalDocsExamined:
12
nReturned: 2
```

Avec index

```
stage: FETCH ← IXSCAN
totalKeysExamined: 2
totalDocsExamined: 2
nReturned: 2
```

Les informations à lire

Champ	Signification
`stage`	**COLLSCAN** tout lire, **IXSCAN** index, **FETCH** lire le document, **SORT** tri en mémoire
`nReturned`	documents renvoyés
`totalKeysExamined`	entrées d'index lues
`totalDocsExamined`	documents lus

i Requête efficace : **totalDocsExamined ≈ nReturned**. Beaucoup plus grand : il manque sans doute un index. L'étape se lit dans **queryPlanner.winningPlan**.

L'agrégation

Produire une information nouvelle à partir des données

L'agrégation calcule un résultat nouveau (nombre, moyenne, classement…) en combinant **plusieurs documents** (étudiants par filière) ou **plusieurs valeurs d'un même document** (compétences d'un étudiant).

find() : retrouver des documents

« Quels sont les étudiants d'Informatique ? »

```
db.etudiants.find(
  { filiere: "Informatique" })
```

```
{ nom: "Sara Alaoui", age: 21, ... }
{ nom: "Yassine El Amrani", ... }
{ nom: "Amine Tazi", age: 23, ... }
// ... 6 fiches complètes
```

→ Les documents **tels qu'ils sont stockés**

Agrégation : calculer un résultat

« Combien d'étudiants dans chaque filière ? »

```
db.etudiants.aggregate([
  { $group: { _id: "$filiere",
              nb: { $sum: 1 } } } ])
```

```
{ _id: "Informatique",   nb: 6 }
{ _id: "Data Science",   nb: 2 }
{ _id: "Réseaux",        nb: 2 }
{ _id: "Génie Logiciel", nb: 1 }
```

→ Une **information nouvelle**, absente des documents

L'agrégation répond à « combien ? », « quelle moyenne ? », « qui est en tête ? » (en SQL : **COUNT**, **AVG**, **GROUP BY**). Dans MongoDB, elle s'écrit avec **aggregate()**, sous forme de **pipeline**.

L'étape $project

Choisir, renommer et calculer des champs, document par document

$project traite les documents un par un : il garde, masque ou **calcule** des champs. Le nombre de documents ne change pas, contrairement à **$group** (11 étudiants → 4 filières).

Nombre de compétences par étudiant

```
db.etudiants.aggregate([
  { $project: {
      _id: 0,       // masquer
      nom: 1,       // garder
      nbCompetences: // calculer
        { $size: "$competences" }
  } } ])
```

```
{ nom: "Sara Alaoui", nbCompetences: 3 }
{ nom: "Omar Fassi", nbCompetences: 2 }
// ... 11 résultats, un par étudiant
```

Ce qu'on écrit dans $project

Écriture	Effet
`champ: 1`	garder le champ
`champ: 0`	masquer le champ
`nouveau: "$champ"`	copier / renommer
`{ $size: "$competences" }`	**nombre** d'éléments
`{ $concat: ["$nom", "$ville"] }`	**texte** assemblé
`{ $cond: [ condition, valeurSiVrai, valeurSiFaux ] }`	**si… alors… sinon**

Pas de mélange de `1` et `0` (sauf `_id`). Tout garder : `$addFields`.

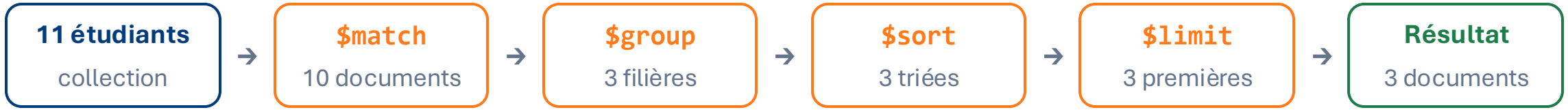
```
{ $merge: { into: "etudiants", on: "_id",
            whenMatched: "merge" } }
```

Pour enregistrer le champ : **$merge** en dernière étape, sans `_id: 0`. Rien n'est alors affiché.

Le pipeline d'agrégation

Calculer des indicateurs à partir de plusieurs documents

Un pipeline est une chaîne d'étapes. Les documents passent d'une étape à l'autre : chacune transforme ce qu'elle reçoit et transmet **son résultat** à la suivante.



Les 3 filières avec le plus d'étudiants ≥ 12

```
db.etudiants.aggregate([
  { $match: { moyenne: { $gte: 12 } } },
  { $group: {
      _id: "$filiere", nb: { $sum: 1 },
      moyenne: { $avg: "$moyenne" } } },
  { $sort: { nb: -1, moyenne: -1 } },
  { $limit: 3 } ])
```

- **1. $match**
  garde les moyennes ≥ 12 : Mehdi est retiré.
- **2. $group**
  un document par filière, avec nombre et moyenne.
- **3. $sort**
  par nombre décroissant, puis par meilleure moyenne.
- **4. $limit**
  garde les 3 premières filières.

Calculs possibles dans $group : **$sum** compter ou additionner · **$avg** moyenne · **$min** / **$max** plus petite / plus grande
$push liste des valeurs · **$addToSet** liste sans doublon · **$first** / **$last** première / dernière valeur du groupe

Les 3 niveaux d'une requête d'agrégation

Étape, accumulateur, expression : où s'écrit chaque opérateur

Total des compétences par filière

```
db.etudiants.aggregate([
  { $group: {                  // 1. ÉTAPE
      _id: "$filiere",
      total: { $sum:           // 2. ACCUMULATEUR
        { $size: "$competences" } // 3. EXPRESSION
  } } } ])
```

Lire de l'intérieur vers l'extérieur

3. `$size` : compte les compétences **d'un étudiant**
2. `$sum` : additionne ces nombres pour **tous les étudiants du groupe**
1. `$group` : produit **un document par filière**

Niveau	Où l'écrit-on ? Que fait-il ?	Opérateurs courants
1. Étape	dans le pipeline `[ ]` : transforme le flux	`$match $project $group $sort $unwind $lookup`
2. Accumulateur	dans `$group` : résume **un groupe**	`$sum $avg $min $max $push $addToSet`
3. Expression	dans une étape : agit sur **un document**	`$size $concat $cond $multiply $round $gte`

¡ **Piège : `$gte`** s'écrit `{ moyenne: { $gte: 12 } }` dans un filtre, mais `{ $gte: ["$moyenne", 12] }` comme **expression** → c'est le rôle de **`$expr`**.

L'opérateur $expr

Utiliser les expressions d'agrégation dans un filtre

Méthode classique

Compare un champ à une **valeur fixe**

```
db.etudiants.find({
  moyenne: { $gte: 12 } })
```

✓ Simple et peut utiliser un index

Avec $expr

Opérateur **d'abord**, puis un tableau **[a, b]**

```
db.etudiants.find({
  $expr: { $gte: ["$moyenne", 12] } })
```

"$moyenne" = la valeur du champ moyenne

Son intérêt : comparer deux champs du même document

```
// Moyenne en baisse depuis le S1
db.etudiants.find({
  $expr: {
    $lt: ["$moyenne", "$moyenneS1"]
} })
```

```
// Étudiants en baisse par filière
db.etudiants.aggregate([
  { $match: { $expr: {
      $lt: ["$moyenne", "$moyenneS1"] } } },
  { $group: { _id: "$filiere",
      nb: { $sum: 1 } } } ])
```

i | Sans **$expr**, **"$moyenneS1"** est lu comme du texte · **valeur fixe** → classique · **deux champs** → **$expr**

Tableaux et jointures : $unwind et $lookup

Analyser les tableaux imbriqués, joindre les collections référencées

$unwind : compter par compétence

Un **$group** sur un tableau regroupe le **tableau entier**.

```
db.etudiants.aggregate([
  { $unwind: "$competences" },
  { $group: { _id: "$competences",
              nb: { $sum: 1 } } },
  { $sort: { nb: -1 } }
])
```

```
{ _id: "MongoDB", nb: 5 }
{ _id: "Python",  nb: 5 }
{ _id: "Java",    nb: 4 }  // ...
```

Sara → **3 documents**, un par compétence.

$lookup : récupérer la formation

L'étudiant ne contient que **formationId: 10**.

```
db.etudiants.aggregate([
  { $lookup: { from: "formations",
      localField: "formationId",
      foreignField: "_id",
      as: "formation" } },
  { $unwind: "$formation" },
  { $project: { _id: 0, nom: 1,
      formation: "$formation.nom" } } ])
```

```
{ nom: "Sara Alaoui",
  formation: "Génie logiciel" }
// ... un document par étudiant
```

$lookup renvoie **toujours un tableau** → **$unwind**.

¡ **Imbriqué** (compétences, notes) → **$unwind** pour analyser · **Référencé** (formation, module) → **$lookup** pour joindre, comme **JOIN** en SQL.

Calculs et recherche textuelle

Calculer entre champs, arrondir, regrouper tout, chercher des mots

Moyenne pondérée de chaque étudiant

```
db.etudiants.aggregate([
  { $unwind: "$notes" },
  { $group: { _id: "$nom",
      points: { $sum: { $multiply:
          ["$notes.note", "$notes.coef"] } },
      coefs: { $sum: "$notes.coef" } } },
  { $project: { moyenne: { $round: [
      { $divide: ["$points", "$coefs"] },
      2 ] } } }
])
```

- **$multiply**
  multiplie deux valeurs : **`note × coef`**, calculé pour chaque note avant la somme.
- **$divide**
  divise la somme des points par la somme des coefficients.
- **$round**
  `[valeur, 2]` arrondit à 2 décimales.
- **_id: null**
  un **seul groupe** pour toute la collection : **`{ $group: { _id: null, nb: { $sum: 1 } } }`**

Recherche textuelle

```
db.etudiants.createIndex({ projet: "text" })
db.etudiants.find({ $text: {
    $search: "mongodb sécurité" } })
```

- Cherche des **mots entiers** (casse ignorée).
- Plusieurs mots : l'un **ou** l'autre.
- **Un seul** index texte par collection.

Valider la structure avec JSON Schema

Refuser les documents qui ne respectent pas les règles

Règles pour la collection etudiants

```
db.runCommand({ collMod: "etudiants",
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nom", "filiere", "moyenne"],
    properties: {
      nom: { bsonType: "string" },
      filiere: { enum: ["Informatique",
        "Data Science", "Réseaux"] },
      moyenne: { bsonType: "number",
        minimum: 0, maximum: 20 },
      competences: { bsonType: "array",
        items: { bsonType: "string" } }
} } } })
```

Les mots-clés

Mot-clé	Rôle
`bsonType`	type attendu : string, number, date, array, object
`required`	champs obligatoires
`properties`	règles de chaque champ
`enum`	liste des valeurs autorisées
`minimum maximum`	bornes d'un nombre
`items`	règle pour chaque élément d'un tableau
`minItems`	taille minimale d'un tableau

`collMod` : collection existante ; `createCollection` : nouvelle. Les champs non cités restent **libres**.

Tester et régler la validation

Ce qui se passe quand une règle n'est pas respectée

Insertion refusée

```
db.etudiants.insertOne({
  nom: "Omar", filiere: "Médecine",
  moyenne: 25 })
MongoServerError:
Document failed validation
```

Avertir au lieu de refuser

```
db.runCommand({ collMod: "etudiants",
  validationAction: "warn",
  validationLevel: "moderate" })
```

- **Pourquoi refusé ?**
  « Médecine » n'est pas dans **`enum`**, et 25 dépasse **`maximum: 20`**. Le détail est donné dans **`errInfo`**.
- **validationAction**
  `"error"` refuse le document (par défaut) ; **`"warn"`** l'accepte et écrit un avertissement dans les journaux.
- **validationLevel**
  `"strict"` vérifie tout (par défaut) ; **`"moderate"`** ne vérifie pas les modifications des documents **déjà invalides**.
- **Voir la règle active**
  `db.getCollectionInfos()` affiche le validateur de chaque collection.

TP — Application

Optimiser, analyser et sécuriser la collection etudiants

? **Mme Bennani :** *« La plateforme doit rester rapide, me donner des statistiques et refuser les erreurs. »*

Index et explain()

1. Créez un index sur **`filiere`**, puis affichez les index de la collection.
2. Avec **`explain("executionStats")`**, comparez la recherche des étudiants d'Informatique **avant** et **après** l'index : étape, **`totalDocsExamined`**, **`nReturned`**.
3. Créez l'index **`{ filiere: 1, moyenne: -1 }`** et vérifiez qu'il sert pour : les étudiants d'Informatique **triés par moyenne décroissante**.

Agrégation et validation

4. Le **nombre d'étudiants** et la **moyenne** de chaque filière, de la meilleure à la moins bonne.
5. La **moyenne générale** de toute la promotion (un seul résultat).
6. Les **3 compétences les plus répandues** chez les étudiants d'Informatique.
7. Le nombre d'étudiants **par ville**, parmi ceux qui ont au moins 14.
8. Une validation : **`nom`** obligatoire, **`age`** entre 18 et 30 et **au moins une compétence**. Testez-la avec un document **invalide**.